



UNIVERZITET U NOVOM SADU FAKULTET TEHNIČKIH NAUKA



UNIVERZITET U NOVOM SADU
FAKULTET TEHNIČKIH NAUKA
NOVI SAD
Departman za računarstvo i automatiku
Odsek za računarsku tehniku i računarske komunikacije

PROJEKTNI ZADATAK

Kandidat: Filip Cvejić
Broj indeksa: RA182/2024

Predmet: Osnove paralelnog programiranja i softverski alati
Tema rada: MAVN prevodilac

Novi Sad, jun, 2026.

SADRŽAJ

1. Uvod.....	4
1.1 MAVN – viši MIPS32 asemblerski jezik.....	4
1.2 Zadatak.....	5
2. Analiza problema.....	6
3. Koncept rešenja.....	7
4. Opis rešenja.....	9
4.1 Leksička analiza (LexicalAnalysis).....	9
4.1.1 Metoda za pokretanje leksičke analize (Do).....	9
4.1.2 Metoda za dobavljanje sledećeg tokena (getNextTokenLex).....	9
4.2 Sintaksna analiza (SyntaxAnalysis).....	9
4.2.1 Metoda za pokretanje sintaksne analize (Do).....	9
4.2.2 Metode neterminala (Q, S, L, E).....	9
4.2.3 Metoda za proveru tokena (eat).....	10
4.2.4 Metoda za gledanje sledećeg tokena (peekNextTokenType).....	10
4.2.5 Metoda za registarske promenljive (getOrCreateReg).....	10
4.2.6 Metoda za kreiranje instrukcije (emit).....	10
4.2.7 Metoda za izgradnju grafa toka kontrole (buildControlFlowGraph).....	11
4.3 Međureprezentacija (Variable i Instruction).....	11
4.3.1 Metoda za asemblerski oblik instrukcije (toString).....	11
4.4 Analiza životnog veka (LivenessAnalysis).....	11
4.4.1 Funkcija za analizu životnog veka (doLivenessAnalysis).....	12
4.5 Graf smetnji (InterferenceGraph).....	12
4.5.1 Funkcija za formiranje grafa smetnji (doInterferenceGraph).....	12
4.5.2 Funkcija za ispis grafa smetnji (printInterferenceGraph).....	13
4.5.3 Funkcija za oslobađanje grafa smetnji (freeInterferenceGraph).....	13
4.6 Uprošćavanje (Simplification).....	13
4.6.1 Funkcija za uprošćavanje (doSimplification).....	13
4.7 Alokacija resursa (ResourceAllocation).....	13
4.7.1 Funkcija za dodelu registara (doResourceAllocation).....	13
4.7.2 Funkcija za proveru dodele (checkResourceAllocation).....	14
4.8 Generisanje koda (main).....	14

5. Verifikacija i testiranje.....	15
5.1 Test primeri.....	15
5.2 Rezultati.....	15
6. Uputstvo za pokretanje.....	16
7. Zaključak.....	17

1. Uvod

1.1 MAVN – viši MIPS32 asemblerski jezik

MAVN (MIPS Assembler Visokog Nivoa) je viši MIPS 32-bitni asemblerski jezik koji uvodi koncept registarske promenljive. Umesto konkretnih procesorskih registara, programer u instrukcijama koristi registarske promenljive i memorijske promenljive. Time se programiranje pojednostavljuje jer programer ne vodi računa o tome koji su fizički registri zauzeti niti o njihovom sadržaju.

Memorijska promenljiva deklarise se naredbom `_mem` (ime počinje slovom `m` i nosi početnu vrednost), registarska naredbom `_reg` (ime počinje slovom `r`), a funkcija naredbom `_func`. MAVN podržava deset osnovnih instrukcija, prikazanih u Tabeli 1, kojima su u ovom radu dodate još tri instrukcije (`and`, `or` i `bgez`).

Tabela 1 Instrukcije MAVN jezika

Instrukcija	Oblik	Značenje
<code>add</code>	<code>add rid, rid, rid</code>	sabiranje dva registra
<code>addi</code>	<code>addi rid, rid, num</code>	sabiranje registra sa konstantom
<code>sub</code>	<code>sub rid, rid, rid</code>	oduzimanje dva registra
<code>la</code>	<code>la rid, mid</code>	učitavanje adrese memorijske promenljive
<code>li</code>	<code>li rid, num</code>	učitavanje konstante u registar
<code>lw</code>	<code>lw rid, num(rid)</code>	čitanje reči iz memorije
<code>sw</code>	<code>sw rid, num(rid)</code>	upis reči u memoriju
<code>b</code>	<code>b id</code>	bezuslovni skok na labelu
<code>bltz</code>	<code>bltz rid, id</code>	skok ako je registar manji od nule
<code>nop</code>	<code>nop</code>	instrukcija bez operacije
<code>and</code>	<code>and rid, rid, rid</code>	logičko I (dodatna)
<code>or</code>	<code>or rid, rid, rid</code>	logičko ILI (dodatna)
<code>bgez</code>	<code>bgez rid, id</code>	skok ako je registar veći ili jednak nuli (dodatna)

Sintaksa jezika opisana je sledećom gramatikom:

$Q \rightarrow S ; L$

$L \rightarrow \text{eof} \mid Q$

S -> _mem mid num | _reg rid | _func id | id : E | E
E -> add rid,rid,rid | addi rid,rid,num | sub rid,rid,rid
| la rid,mid | lw rid,num(rid) | li rid,num | sw rid,num(rid)
| b id | bltz rid,id | nop

1.2 Zadatak

Zadatak je realizovati MAVN prevodilac koji učitano datoteku sa ekstenzijom .mavn prevodi na osnovni MIPS 32-bitni asemblerski jezik. Prevodilac prilikom prevođenja treba da:

- izvrši leksičku i sintaksnu analizu i prijavi leksičke i sintaksne greške,
- izvrši analizu životnog veka promenljivih,
- formira graf smetnji,
- dodeli registre koristeći najviše četiri registra (t0, t1, t2 i t3),
- generiše ispravan MIPS 32-bitni asemblerski kod u izlaznu datoteku sa ekstenzijom .s.

Pored deset osnovnih instrukcija, potrebno je realizovati još najmanje tri instrukcije iz skupa MIPS instrukcija, od kojih najmanje jedna nije iz aritmetičko-logičke grupe.

2. Analiza problema

Zadatak se svodi na to da program pisan pomoću registarskih promenljivih prevedemo u program koji barata konačnim brojem fizičkih registara. Na raspolaganju su četiri registra (t_0 , t_1 , t_2 i t_3), a ulazni program može imati proizvoljno mnogo registarskih promenljivih. Otuda i glavni problem: kako te promenljive rasporediti na svega četiri registra.

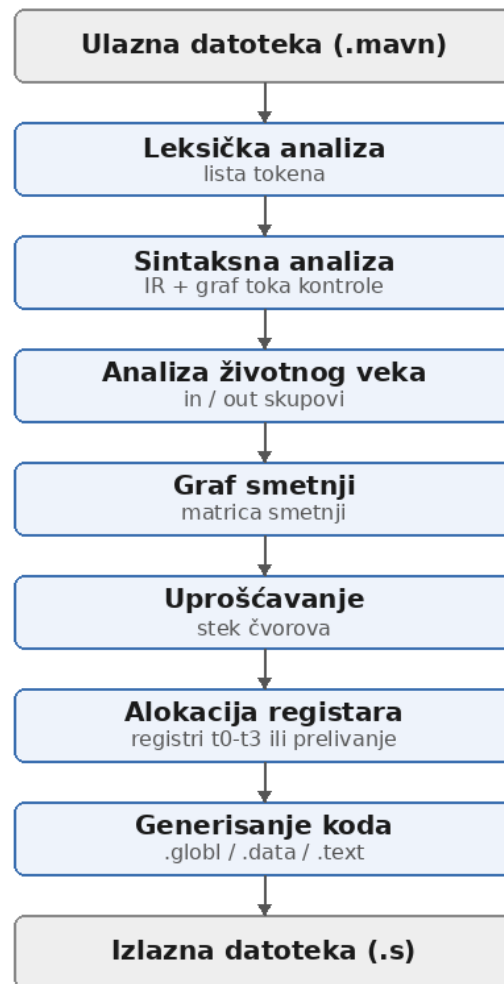
Da bi dodela bila ispravna, mora se znati koje su promenljive istovremeno žive, jer dve istovremeno žive promenljive ne smeju da dele isti registar. Ovo zahteva analizu životnog veka promenljivih i formiranje grafa smetnji, a zatim bojenje tog grafa sa najviše četiri boje, gde svaka boja odgovara jednom registru.

Pored dodele registara, prevodilac mora da prepozna i prijavi greške u ulaznom programu. Leksička greška nastaje kada se naiđe na znak koji ne pripada jeziku, a sintaksna greška kada redosled tokena ne odgovara gramatici. Izlaz mora biti sintaksno ispravan MIPS 32-bitni asemblerski kod.

Kada program u nekom trenutku ima više od četiri žive promenljive, graf smetnji se ne može obojiti sa četiri boje. To je prelivanje (*spill*); prevodilac ga prepoznaje i prijavljuje.

3. Koncept rešenja

Prevodilac je podeljen na faze koje se nižu jedna za drugom, tako da ono što jedna faza proizvede ulazi u sledeću. Pregled faza dat je na Slici 1.



Slika 1 Faze MAVN prevodioca

Leksička analiza deli ulazni tekst na tokene pomoću konačnog automata. Izlaz ove faze je lista tokena.

Sintaksna analiza proverava da li niz tokena odgovara gramatici jezika i prijavljuje sintaksne greške. Tokom ove faze istovremeno se gradi međureprezentacija programa.

Međureprezentacija predstavlja program kao listu instrukcija. Svaka instrukcija nosi svoj asemblerski šablon i liste odredišnih i izvornih registarskih promenljivih. U ovoj fazi formira se i graf toka kontrole, u kome svaka instrukcija ima svoje sledbenike i prethodnike.

Analiza životnog veka za svaku instrukciju određuje skup promenljivih živih na ulazu i na izlazu instrukcije, na osnovu grafa toka kontrole.

Graf smetnji nastaje iz rezultata analize životnog veka i čuva se kao matrica smetnji. Dve promenljive su u smetnji kada su u istom trenutku žive, pa ne smeju dobiti isti registar.

Alokacija registara uprošćava graf smetnji i boji ga sa najviše četiri boje (t0, t1, t2 i t3). Ako bojenje sa četiri boje nije moguće, prijavljuje se prelivanje.

Generisanje koda upisuje izlaznu datoteku sa ekstenzijom .s. Datoteka sadrži sekcije .globl, .data i .text, a registarske promenljive zamenjuju se dodeljenim registrima.

Centralne strukture podataka su *Variable*, koja predstavlja jednu promenljivu (registarsku ili memorijsku), i *Instruction*, koja predstavlja jednu instrukciju zajedno sa pripadajućim promenljivama i vezama u grafu toka kontrole.

4. Opis rešenja

4.1 Leksička analiza (LexicalAnalysis)

Leksičku analizu obavlja konačni automat, kroz klase *FiniteStateMachine*, *LexicalAnalysis* i *Token* iz priložene osnove. Ulazni tekst se razbija na tokene, a dobijena lista tokena potom ide u sintaksnu analizu.

4.1.1 Metoda za pokretanje leksičke analize (Do)

bool Do();

Prolazi kroz ulazni bafer i formira listu tokena, pri čemu beline preskače. Ako naiđe na znak koji ne pripada jeziku, formira token greške i prekida analizu.

Povratne vrednosti:

- true – nema leksičke greške,
- false – pronađena leksička greška.

4.1.2 Metoda za dobavljanje sledećeg tokena (getNextTokenLex)

Token getNextTokenLex();

Pomoću konačnog automata prepoznaje i vraća sledeći token iz ulaznog bafera.

4.2 Sintaksna analiza (SyntaxAnalysis)

Sintaksnu analizu radimo rekursivnim spustom: svaki neterminal gramatike ima svoju metodu. Osim što proverava gramatiku i javlja sintaksne greške, ova faza usput gradi i međureprezentaciju i graf toka kontrole.

4.2.1 Metoda za pokretanje sintaksne analize (Do)

bool Do();

Pokreće analizu od početnog neterminala i, ako je program ispravan, gradi međureprezentaciju i graf toka kontrole.

Povratne vrednosti:

- true – nema sintaksne greške,
- false – pronađena sintaksna greška.

4.2.2 Metode neterminala (Q, S, L, E)

void Q(); void S(); void L(); void E();

Svaka metoda odgovara jednom neterminalu gramatike i proverava da li redosled tokena odgovara produkcijama tog neterminala. Q je početni neterminal, S je jedna naredba, L je nastavak programa, a E je jedna instrukcija.

4.2.3 Metoda za proveru tokena (eat)

```
void eat(TokenType t);
```

Ako je tekući token očekivanog tipa, prelazi na sledeći token; u suprotnom prijavljuje sintaksnu grešku.

Parametri:

- t – očekivani tip tokena.

4.2.4 Metoda za gledanje sledećeg tokena (peekNextTokenType)

```
TokenType peekNextTokenType();
```

Vraća tip sledećeg tokena bez njegovog konzumiranja. Koristi se da bi se razlikovala labela (iza koje sledi dvotačka) od dodatne instrukcije, jer obe počinju identifikatorom.

Povratne vrednosti:

- tip sledećeg tokena (komentari se preskaču).

4.2.5 Metoda za registarske promenljive (getOrCreateReg)

```
Variable* getOrCreateReg(string name);
```

Vraća registarsku promenljivu sa datim imenom, a ako ne postoji, kreira je. Time se obezbeđuje da se za isto ime svuda koristi isti objekat promenljive.

Parametri:

- name – ime registarske promenljive.

Povratne vrednosti:

- pokazivač na promenljivu.

4.2.6 Metoda za kreiranje instrukcije (emit)

```
void emit(InstructionType type, string asmTemplate, Variable* dst, Variable* src1, Variable* src2);
```

Kreira instrukciju zadatog tipa sa asemblerskim šablonom i puni liste odredišnih i izvornih registara. Odredišni registar ulazi u skup def, a izvorni u skup use, što je ulaz za analizu životnog veka.

Parametri:

- type – tip instrukcije,
- asmTemplate – asemblerski šablon sa oznakama `d i `s,

- dst – odredišni registar (ili nijedan),
- src1, src2 – izvorni registri (ili nijedan).

4.2.7 Metoda za izgradnju grafa toka kontrole (buildControlFlowGraph)

void buildControlFlowGraph();

Za svaku instrukciju postavlja sledbenike i prethodnike. Obična instrukcija vodi na sledeću instrukciju, uslovni skok vodi i na sledeću i na ciljnu, a bezuslovni skok samo na ciljnu. Skok na nepostojeću labelu prijavljuje se kao greška.

4.3 Međureprezentacija (Variable i Instruction)

Program se interno predstavlja listom instrukcija. Struktura *Variable* predstavlja jednu promenljivu, a *Instruction* jednu instrukciju.

Variable sadrži tip (registarska ili memorijska), ime, poziciju (indeks u matrici smetnji), dodeljeni registar i, za memorijske promenljive, početnu vrednost.

Instruction sadrži tip, asemblerski šablon, liste odredišnih (dst) i izvornih (src) registara, skupove use, def, in i out, kao i liste sledbenika i prethodnika u grafu toka kontrole.

4.3.1 Metoda za asemblerski oblik instrukcije (toString)

string toString();

Popunjava asemblerski šablon stvarnim registrima: oznaku `d zamenjuje odredišnim, a `s izvornim registrom, redom kojim se pojavljuju. Koristi se pri generisanju koda.

Povratne vrednosti:

- asemblerski oblik instrukcije sa dodeljenim registrima.

4.4 Analiza životnog veka (LivenessAnalysis)

Analiza životnog veka određuje, za svaku instrukciju, koje su promenljive žive na ulazu i na izlazu instrukcije.

Tabela 2 prikazuje rezultat analize životnog veka za program simple.mavn.

Tabela 2 Analiza životnog veka za program simple.mavn

Pozicija	Instrukcija	IN	OUT
1	la r4, m1	{ }	{ r4 }
2	lw r1, 0(r4)	{ r4 }	{ r1 }
3	la r5, m2	{ r1 }	{ r1, r5 }
4	lw r2, 0(r5)	{ r1, r5 }	{ r1, r2 }
5	add r3, r1, r2	{ r1, r2 }	{ }

4.4.1 Funkcija za analizu životnog veka (doLivenessAnalysis)

```
void doLivenessAnalysis(Instructions& instructions);
```

Iterativno, unazad, računa skupove in i out dok se vrednosti ne ustale (do fiksne tačke). Za svaku instrukciju važi: out je unija in skupova svih sledbenika, a in je unija skupa use i razlike skupova out i def.

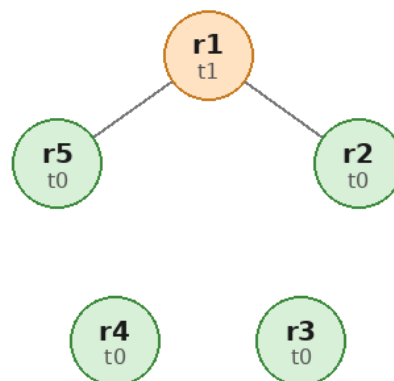
Parametri:

- instructions – lista instrukcija sa popunjenim use, def i grafom toka kontrole.

4.5 Graf smetnji (InterferenceGraph)

Graf smetnji se formira iz rezultata analize životnog veka i predstavlja se matricom smetnji. Dve promenljive su u smetnji ako su istovremeno žive i tada ne smeju da dele isti registar.

Slika 2 prikazuje graf smetnji za program simple.mavn. Promenljiva r1 je u smetnji sa r2 i r5, pa dobija drugi registar (t1) u odnosu na njih, dok promenljive r3 i r4 nemaju smetnji i mogu da koriste isti registar (t0).



Slika 2 Graf smetnji za program simple.mavn

4.5.1 Funkcija za formiranje grafa smetnji (doInterferenceGraph)

```
InterferenceGraph* doInterferenceGraph(Instructions& instructions, Variables& registerVariables);
```

Formira matricu smetnji. Za svaku instrukciju, promenljiva koja se definiše stavlja se u smetnju sa svim promenljivama koje su žive na izlazu te instrukcije.

Parametri:

- instructions – lista instrukcija sa rezultatom analize životnog veka,
- registerVariables – lista registarskih promenljivih.

Povratne vrednosti:

- pokazivač na formirani graf smetnji.

4.5.2 Funkcija za ispis grafa smetnji (printInterferenceGraph)

```
void printInterferenceGraph(InterferenceGraph* ig);
```

Ispisuje matricu smetnji radi provere.

Parametri:

- ig – graf smetnji.

4.5.3 Funkcija za oslobađanje grafa smetnji (freeInterferenceGraph)

```
void freeInterferenceGraph(InterferenceGraph* ig);
```

Oslobađa memoriju zauzetu matricom smetnji.

Parametri:

- ig – graf smetnji.

4.6 Uprošćavanje (Simplification)

Uprošćavanje priprema graf smetnji za bojenje uklanjanjem čvorova niskog stepena.

4.6.1 Funkcija za uprošćavanje (doSimplification)

```
stack<Variable*>* doSimplification(InterferenceGraph* ig, int degree);
```

Sukcesivno uklanja čvor čiji je stepen manji od broja registara i stavlja ga na stek. Ako u nekom trenutku nijedan preostali čvor nema stepen manji od broja registara, javlja se prelivanje.

Parametri:

- ig – graf smetnji,
- degree – broj raspoloživih registara.

Povratne vrednosti:

- stek čvorova za bojenje, ili prazan pokazivač ako je došlo do prelivanja.

4.7 Alokacija resursa (ResourceAllocation)

Alokacija resursa dodeljuje registre (boje) promenljivama na osnovu steka iz faze uprošćavanja.

4.7.1 Funkcija za dodelu registara (doResourceAllocation)

```
bool doResourceAllocation(stack<Variable*>* simplificationStack, InterferenceGraph* ig);
```

Skida čvorove sa steka i svakom dodeljuje prvi registar koji ne koristi nijedan sused u smetnji. Ako neki čvor ne može da dobije registar, javlja se stvarno prelivanje.

Parametri:

- simplificationStack – stek čvorova iz faze uprošćavanja,
- ig – graf smetnji.

Povratne vrednosti:

- true – uspešna dodela,
- false – stvarno prelivanje.

4.7.2 Funkcija za proveru dodele (checkResourceAllocation)

bool checkResourceAllocation(InterferenceGraph* ig);

Proverava da nijedne dve promenljive koje su u smetnji nemaju isti registar.

Parametri:

- ig – graf smetnji.

Povratne vrednosti:

- true – dodela je korektna,
- false – dve promenljive u smetnji dele isti registar.

4.8 Generisanje koda (main)

Generisanje koda upisuje izlaznu datoteku sa ekstenzijom .s. Najpre se upisuje ime funkcije kao globalni simbol (.globl), zatim sekcija podataka (.data) sa memorijskim promenljivama, pa programska sekcija (.text). Za svaku instrukciju poziva se *toString*, koji u šablon ubacuje dodeljene registre, a ispred instrukcije koja nosi labelu upisuje se ta labela.

5. Verifikacija i testiranje

Rad prevodioca proveren je na skupu test primera u MAVN jeziku. Za svaki primer gleda se da li je ponašanje očekivano: ispravan program treba da da izlaznu .s datoteku, neispravan da prijavi grešku, a program sa više od četiri žive promenljive da prijavi preliivanje.

5.1 Test primeri

Korišćeni su sledeći test primeri:

- simple.mavn – osnovni primer iz postavke (la, lw, add),
- test_base.mavn – osnovne instrukcije (la, lw, li, add, sub, addi, sw, nop),
- test_loop.mavn – petlja sa labelama i skokovima (b, bltz),
- test_extra.mavn – dodatne instrukcije (and, or, bgez),
- multiply.mavn – primer sa više od četiri istovremeno žive promenljive,
- err_lexical.mavn – nedozvoljen znak u programu,
- err_syntax.mavn – instrukcija sa nedostajućim operandom,
- err_label.mavn – skok na nepostojeću labelu.

5.2 Rezultati

Svi test primeri dali su očekivano ponašanje, što je prikazano u Tabeli 3.

Tabela 3 Rezultati testiranja

Test primer	Šta se proverava	Ishod
simple.mavn	prevođenje osnovnog primera	generisana .s datoteka
test_base.mavn	osnovne instrukcije	generisana .s datoteka
test_loop.mavn	petlja, labele i skokovi	generisana .s datoteka
test_extra.mavn	dodatne instrukcije	generisana .s datoteka
multiply.mavn	pritisak veći od četiri registra	prijavljeno preliivanje
err_lexical.mavn	detekcija leksičke greške	prijavljena leksička greška
err_syntax.mavn	detekcija sintaksne greške	prijavljena sintaksna greška
err_label.mavn	skok na nepostojeću labelu	prijavljena greška

Generisane .s datoteke predstavljaju sintaksno ispravan MIPS 32-bitni asemblerski kod i mogu se izvršiti u simulatoru QtSpim.

6. Uputstvo za pokretanje

Projekat se prevodi i pokreće u razvojnom okruženju Visual Studio. Otvara se datoteka rešenja *LexicalAnalysis.sln*, a zatim se projekat prevodi opcijom Build.

Program kao argument komandne linije prima putanju do ulazne datoteke sa ekstenzijom *.mavn*. Ako argument nije zadat, koristi se podrazumevani primer *examples/simple.mavn*. Izlazna datoteka dobija isto ime kao ulazna, sa ekstenzijom *.s*, i smešta se pored ulazne datoteke.

Primer pokretanja:

`LexicalAnalysis.exe examples\simple.mavn`

Nakon uspešnog prevođenja, dobijena datoteka *examples/simple.s* može se učitati i izvršiti u simulatoru QtSpim.

7. Zaključak

U radu je realizovan MAVN prevodilac koji program sa višeg MIPS 32-bitnog asemblerskog jezika prevodi na osnovni MIPS 32-bitni assembler. Obuhvaćene su sve tražene faze: od leksičke i sintaksne analize, preko analize životnog veka, grafa smetnji i alokacije registara, do generisanja koda.

Prevodilac detektuje i prijavljuje leksičke i sintaksne greške, kao i prelivanje kada program zahteva više od četiri istovremeno žive promenljive. Pored deset osnovnih instrukcija, implementirane su i tri dodatne instrukcije (and, or i bgez), od kojih bgez nije iz aritmetičko-logičke grupe.

Rešenje je provereno nad skupom test primera koji pokrivaju osnovne i dodatne instrukcije, petlje, detekciju grešaka i prelivanje. Generisani asemblerski kod je ispravan i može se izvršiti u simulatoru QtSpim.